

Doom Computation: Abstract Obstructions and a Bounded Rule110 Fallback

Project21 research artifacts

Abstract

This is a negative/partial research report, not a Doom completeness proof. Core Lean checks an abstract finite-capacity obstruction to lossless encoding, an obstruction to exact fixed-clock embedding of a colliding transition into an injective host, and a finite Rule110 truth table and trace. No universal Turing machine (UTM), native WAD computation, or formal C refinement is constructed. The finite-state obstruction has a specified direction and does not refute an unrestricted existential universality claim.

1 Scope and transition evidence

We inspected Chocolate Doom’s `src/doom/g_game.c` and `src/doom/p_tick.c` at reference commit

895f581c5d91497bdda0516612da803fe5843e28.

The two files were clean relative to that checkout. In `g_game.c`, lines 910–1081, `G_Ticker` handles reborns and pending game actions, consumes player commands (with demo playback/recording branches), handles special buttons, and dispatches by game state. In `p_tick.c`, lines 125–153, `P_Ticker` can return on pause/menu conditions; otherwise it updates players, thinkers, specials and respawns, then increments level time. Lines 46–117 show ordered linked-list thinkers and lazy removal. These are source observations, not synchronous cellular-automaton semantics. Pinned links and fuller caveats are in `docs/doom_transition.md`.

A proposed boundary semantics would expose state, commands/environment inputs, and completion, termination or fault outcomes. A total autonomous transition requires a deterministic closure of the environment and explicit handling of partial behavior. A sufficient state must account for globals/statics, heap and ordered thinkers, RNG state, timing, commands and

relevant I/O, not merely a screen image. No complete record or C semantics is formalized. In particular, allocation and growable demo buffers cannot be assumed bounded from these two files alone. No numerical bound on Doom’s state space is proved.

2 Checked abstract obstructions

Let $e : A \rightarrow B$ encode guest states into host states. If a decoder satisfies $d(e(x)) = x$ for all x , then e is injective. Thus a witnessed collision $x \neq y$, $e(x) = e(y)$ excludes such a decoder.

For a duplicate-free list L of source states and a finite list H covering all values of e , injectivity implies $|L| \leq |H|$. Consequently $|H| < |L|$ excludes injection and yields a collision. A source admitting distinct lists longer than every natural bound cannot be losslessly encoded into one fixed finite host cover. These statements, including their explicit cover and unbounded-sample hypotheses, are checked in `proofs/DoomComplete.lean`. They are not applications to an enumerated Doom state space.

Let $g : A \rightarrow A$, $h : B \rightarrow B$, let e be injective, and fix $k \in \mathbb{N}$. If h is injective and

$$h^k(e(x)) = e(g(x)) \quad \text{for all } x,$$

then g is injective: equality of guest successors gives equality of encoded successors; injectivity of h^k and e recovers equality of predecessors. A witnessed collision of g therefore excludes this exact fixed-clock embedding. This also covers $k = 0$ and needs no finiteness assumption. Reversibility is used only through injectivity. Neither injectivity nor a full-state collision of the actual Doom transition is proved here.

3 Why these are not global refutations

The finite-capacity result obstructs lossless encoding of arbitrarily many guest configurations *into* a fixed finite closed host. It does not obstruct encoding a finite Doom system into an unbounded universal machine. A strong existential definition in that opposite direction is not refuted. The reversible obstruction applies only with its extra injectivity and exact fixed-clock hypotheses, not to every possible universal machine. A projected simulation $d(h^k(e(x))) = g(x)$ can retain history in the host and is not the same equation. Restricted states, variable clocks, resource-indexed families and general input/output universality are outside these negative conclusions. No unrestricted impossibility result is asserted.

4 Bounded Rule110 fallback

In `substrate/utm_proof.lean`, the local rule is $(c \& \neg l) | (c \neq r)$ on Booleans. In neighborhood order 111, 110, 101, 100, 011, 010, 001, 000, its outputs are 0, 1, 1, 0, 1, 1, 1, 0. Lean checks agreement with eight explicit pattern cases. The executable row update uses a fixed-length list and false exterior neighbors reset at each step (not a ring or an infinite-domain theorem). Width is preserved and a trace of n updates has $n + 1$ rows. Kernel-checked reduction gives

$$0001000 \rightarrow 0011000 \rightarrow 0111000 \rightarrow 1101000 \rightarrow 1111000.$$

This is four updates at width seven, not a universality proof. A separate conditional lemma merely composes two assumed one-step commuting diagrams; it supplies no implementation or encoding witnesses.

5 Literature and blocked construction

Cook proves universality of Rule110 using a global construction with cyclic tag systems and organized backgrounds/signals [1]. Its background conditions cannot be replaced without proof by our finite zero-exterior row. Neary and Woods prove P-completeness of predicting t steps of Rule110 via polynomial-time deterministic Turing-machine simulation [2]. Neither result supplies a Doom gadget implementation; neither is formalized here. Local truth-table agreement does not establish the global reduction.

`substrate/utm_spec.md` explicitly marks the UTM construction blocked. Missing witnesses include a concrete WAD substrate, a uniform effective input compiler, initialization and invariant proofs, scheduling and memory mechanisms, resource semantics, clock and halt/output decoding, and refinement from C or binary execution. An externally computed command/demo trace is not evidence of native WAD computation. No formal C semantics proof is claimed.

6 Reproducibility and trust

The toolchain file pins `leanprover/lean4:v4.34.0`; the checked compiler reports Lean 4.34.0, Windows x86-64 GNU, commit 293d5d0c0c3f3dded4688b3ccd6a33939ac5102b, Release. Both Lean files import only `Init`; no `mathlib` or package downloads are used for the proof development. From the workspace root, run

```
C:\Users\davea\.elan\bin\lean.exe proofs\DoomComplete.lean
C:\Users\davea\.elan\bin\lean.exe substrate\utm_proof.lean
```

The final commands and exit codes are recorded in `docs/lean_validation.md`. The list-capacity arguments depend on core classical/list infrastructure and the standard foundations `propext`, `Classical.choice`, and `Quot.sound`. The collision/clock algebra and truth-table/trace calculations print no axiom dependencies. There are no custom axioms or admitted proof goals, and no native-decision proof shortcut. The executable evaluations are supplementary; the equalities themselves are checked by the kernel.

7 Executable fallback and empirical evidence

The shipped `doomc` accepts periodic Rule110 JSON specifications and emits a static trace gallery as a vanilla E1M1 PWAD. This boundary convention is different from the formal zero-exterior trace above; no refinement between these two implementations is claimed. For width eight and seven updates, the map has 81 sectors, including entrance corridors. Bright cells encode one and dark cells zero. An exit linedef permits ordinary interactive exploration. The requested filename `doom_utm.wad` is retained, but the artifact is not a UTM. Python computes every cell before Doom starts.

A generated vanilla version-109 demo contains 172 tic commands. Two actual Chocolate Doom 3.1.1 headless playbacks, using Freedoom 0.13.0, write savegames at level time 170. The verifier checks the tic count and all sectors' floor, ceiling, light, special and tag values, then compares a SHA256 projection hash. The two projections match. This is not a full game-state hash, a rendering check, a recorded human playthrough, or evidence of in-engine CA evaluation. Commands and logs accompany the artifacts. Compiler tests cover sizes $N = 1, 2, 4, 8, 16, 32$; $N = 64$ is rejected by explicit backend policy, not by an impossibility theorem or measured engine limit. Tics per simulated CA step are undefined because the engine executes no CA update.

8 Executable PDF: documented limitation

The executable-PDF stretch goal was not implemented. Our deliverable is a static typeset report, not an impossibility theorem about PDF. JavaScript APIs, security policies and rendering facilities vary by viewer. A native executable cannot simply be assumed to run inside a document, but this

does not exclude a JavaScript port, emulator or viewer-specific implementation. No such port or viewer benchmark was attempted here. We provide the PWAD and demo separately. The manuscript can be rebuilt with two invocations of `pdflatex -interaction=nonstopmode -halt-on-error doom_complete.tex` from the paper directory. Local validation used MiKTeX 25.12.

Anticipated Reviewer 2

“Weak completeness is trivial.” Correct, and labelled as such: it appears only as a conditional composition lemma with every premise explicit. *“Where is the universal machine?”* Absent, deliberately: the UTM section enumerates the missing witnesses rather than gesturing at them. *“Your obstruction does not touch the existential claim.”* Also correct, and stated in the theorem’s own direction: it bounds unbounded guests inside a fixed finite closed installation, not Doom inside an unbounded substrate. *“The gallery is not computation.”* It is not offered as computation; each artifact that mentions it says precomputed. *“Prove it end to end in Lean.”* That requires a mechanized C semantics for the engine, which nobody involved has, and which we decline to fake.

9 Conclusion

The delivered fallback is an abstract obstruction library and a bounded Rule110 calculation, not Doom completeness. A native WAD UTM and the semantic bridge to engine execution remain unconstructed. The negative results retain their hypotheses and directions; the broader existential question remains open in this project.

References

- [1] Matthew Cook. Universality in Elementary Cellular Automata. *Complex Systems* 15(1), 1–40, 2004. <https://doi.org/10.25088/ComplexSystems.15.1.1>.
- [2] Turlough Neary and Damien Woods. P-completeness of Cellular Automaton Rule 110. In *Automata, Languages and Programming*, ICALP 2006, LNCS 4051, pp. 132–143. Springer, 2006. https://doi.org/10.1007/11786986_13.

[3] Chocolate Doom contributors. Source revision
895f581c5d91497bdda0516612da803fe5843e28. [https:](https://github.com/chocolate-doom/chocolate-doom/tree/895f581c5d91497bdda0516612da803fe5843e28/src/doom)
[//github.com/chocolate-doom/chocolate-doom/tree/
895f581c5d91497bdda0516612da803fe5843e28/src/doom.](https://github.com/chocolate-doom/chocolate-doom/tree/895f581c5d91497bdda0516612da803fe5843e28/src/doom)